Text Processing with Attention Mechanism

Trang Nguyen

STAT 588 Final Project
Portland State University

July 6, 2026

Outline

1. Motivation for attention from NLP
2. Text processing, Tokenization, sequence, and why attention mechanism is needed
3. Attention mechanism
4. Self-Attention layer in processing sequence of tokens
5. Variational token length and fixed contextual synthesized vector
6. Toy codes and examples of application
7. Summary

Section 1: Motivation for attention from NLP:

Motivation - curiosity

AI and context reading

- Context reading: a fundamental challenge in AI systems (Mitchell, 2019) in her example of reading context in understanding information from a picture
- Modern AI systems: quite strong context-reading abilities; attention mechanisms and transformer models are key.
- Presidents' posts may help drive market direction, but how can text be incorporated into market analysis?

Motivation - specific NLP problem need to be solved

NLP applications and my focus

- Popular natural language processing applications: translation, question answering, sentiment analysis, etc.
- Less directly visible: embedding, represent words/ texts in a geographical space, helping downstream task (including above one) become easier.
- The applied target in my work: text chunk to fixed-length feature vector summarizing the content, ready for use.

Formulation of the problem - 1

- Input: chunk of texts.
  E.g:
  1. "This final presentation series is interesting! It took me long of time to read but it is quite enjoyable."
  2. "This final presentation series is intensive. The course is kind of bringing students to the steep part of the learning curve"
  How is the similarity/ dissimilarity of those two text chunks?
- Output: fixed-length numerical vector that reflects a synthetic meaning, ready to be used for downstream tasks.
- In deep learning model: need some step to make texts become standard numerical data in formulation, especially to make input compatible for computation in attention mechanism.

Section 2: Text processing, Tokenization, sequence, and why attention mechanism is needed:

Tokenization and sequence indexing

- Denotation for input: token and sequence indexing is needed for text chunk.
- A training example is a sequence of **token**, not just words only, potentially include the punctuation, special characters, etc.
- Tokenization process: convert original text into tokens either words, punctuation, symbols, each is a token. All are indexed in a sentence.
- **Example:**
  "This final presentation series is interesting!"
  7 tokens: this, final, presentation, series, is, interesting, !
  $(k_1, k_2, k_3, k_4, k_5, k_6, k_7)$.

Numerical representation of tokens

- Token vector: convert each token to a vector of fixed length.
- This is necessary so that each token has similar standardized format for comparison.
- This can be done by any way: simple word ID, one-hot vector with dictionary/ TF-IDF/trained embedding layer, etc.
- **Summary**: each training example of raw text becomes a variational length sequence of fixed length token vectors.
  $(k_1, k_2, k_3, k_4, k_5, k_6, k_7)$ is converted to a sequence of vector of same length: $(x_1, x_2, x_3, x_4, x_5, x_6, x_7)$.
  A training example become a sequence of numerical representations of tokens: $S_T = (x_1, x_2, \dots x_T)$ where $T = 7$, which can be aslo stacked as a matrix.

Why text is hard to model directly

- Vocabulary size is large, can be 10^4 to 10^6 distinct words.
- In terms of meaning: word order matters, the possible sentence space grows combinatorially. $\rightarrow$ not practical to estimate empirical probability in prediction model.
  Eg: a sentence of length 5 with dictionary size '10^6' has $(10^6)^5$ possible outcomes.
- Key different of text data from usual data: same words, very different meanings depending on their location and other words.
- Sentences link to each other. Context is important in synthesizing.
- **Word within other words is the part that self-attention mechanism addressed, making Transformer model more advanced comparing to other NN**

Section 3: Attention mechanism:

Attention mechanism: search terminologies and intuition

Question: query, key, value what is what?

- Within google search problem:
  - What do you consider keyword as in google search bar?
- Within searching in a spreadsheet:
  - What can be a query in retrieving information from a spreadsheet of student with their ID, name, grade, etc?
  - What do you consider student ID as in this case?
  - What do your consider as values in this spreadsheet?

My confusion stuck in these quite long!

Attention mechanism: search terminologies and intuition

Question: query, key, value what is what?

- In search/ database retrieval:
  - The query Q is what is searching
  - The database is a complete set of pairs of (key, value)=(K,Q)
  - the key (K) part of a database is what is match against with the query (Q) to retrieve useful information from the database.
  - the value (V) is the useful information for being retrieved, what we actually care about

Question: query, key, value what is what?

- Within google search problem: Keyword is within query, different from the terminology **"key"** in data retrieval and attention mechanism.
- Within searching student information in a spreadsheet:
  - Query can be a specific student ID that you're interested in.
  - Set of student ID: each student ID is unique, set of unique for each data row is primary keys in the data retrieval
  - Other information in this database besides the key ID, such as name, grade, etc. are values.

Attention mechanism: summary of search/data retrieval

- Query, key, value:
  - the query Q is what is searching
  - the key (K) is what is matched against the query Q
  - the value (V) is what is retrieved, what we actually care about after the query-key matching process.
  - Database is word in search/ information retrieval; memory/reference is word for attention mechanism. Basically, they are represented as combination of all keys and all values (K,V).
  - Suppose the database/ reference source has T' rows, then the memory is a set of T' key-value pairs: $(K_1, V_1), (K_2, V_2), \ldots (K_{T'}, V_{T'})$.

Attention mechanism: hard search, soft search in data retrieval

- Consider now Q is a set of T queries rather than general name query mentioned previously. A single query is denoted as Q_i
  - **Hard search**:
    - query Q_i match through all Keys in the data base $K_1, K_2, \dots K_{T'}$ as binary: (match, return value V_j) or (no match, return nothing).
    - Final retrieved information is just a single V_j when Q_i match with K_j.
  - **Soft search**:
    - query Q_i match through all possible value of keys in the data base $K_1, K_2, \dots K_{T'}$ as compatible score $C_{i1}, C_{i2}, \dots, C_{iT'}$.
    - This compatible score is then used to construct T' weights $A_{i1}, A_{i2}, \dots, A_{iT'}$ in retrieving all values $V_1, V_2, \dots, V_{T'}$ for query Q_i.
    - Information retrieved is the weighted average value.

Attention mechanism: components and forward pass

- Attention mechanism is a soft search with two inputs: input for query and input for reference/ database.
- The weight $A_{i1}, A_{i2}, \ldots, A_{iT'}$ in retrieving $V_1, V_2, \ldots, V_{T'}$ that soft search is called attention weight for query Q_i.
- in attention mechanism, scaled dot product is usually used to construct compatibility score C_{ij} between query Q_i and key K_j. But the set of function for compatibility is not limited to this.
- Attention weight is computed from compatibility score.
- Any function converting a vector of compatible scores $C_{i1}, C_{i2}, \ldots, C_{iT'}$ to a normalized weight vector can be used as long as $\sum_{j=1}^{T'} A_{ij} = 1$ for all i.
- Final retrieved information from query Q_i is: $(A_{i1}, A_{i2}, \ldots, A_{iT'})V^{T'}$.

Attention mechanism: Q, K, V conversion

- From the input (a single training example and a reference), it construct 3 set of matrices: a set of queries (matrix Q from training example), and the memories contains a set of keys (matrix K) and a set of values (matrix V) from reference.
- Parameter corresponding to each matrix: W_Q, W_K, W_V are learned in training.
- Soft search then is applied to those queries, keys, and values to retrieve useful information from reference corresponding to the training example.
- For example, in self-attention, the training example is also the reference. That is, each component of the training example will be used in a query to retrieve information from all components of the training example.
- This make the long and complicated dependencies happen without loss of information.

Attention mechanism: Scaled dot-product and softmax

Note that there are two function in processing Q, K, V in attention mechanism:

- to compute a compatibility between Q_i and K_j (sometimes mentioned as alignment).
- to compute attention weights from such set of compatibility scores
- In the original Transformer paper, compatibility is the scaled dot-product and after that softmax is used to obtain attention weights.

Attention mechanism: Input for query, key, value

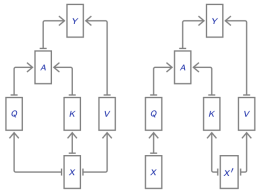
Input:

- Input for query: A training example $S_T = (k_1, \ldots, k_T)$ is pre-processed to be a matrix X of T rows with elements $x_i \in \mathbb{R}^{D_{in}}$.
- Input for key and value (memory/database) comes from training example(s) $S'_{T'} = (k'_1, \ldots, k'_{T'})$ and is pre-processed to be a matrix X' of T' rows with elements $x'_j \in \mathbb{R}^{D_{in}}$.

Complete attention layer in neural network: graphical illustration

A standard attention layer takes as input two sequences X and X' and computes the tensors K, V, and Q as per-row linear functions.

$$Q = XW^{Q\top}$$
$$K = X'W^{K\top}$$
$$V = X'W^{V\top}$$
$$A = \text{softmax}_{row}\left(\frac{QK^\top}{\sqrt{D}}\right)$$
$$Y = AV$$



When $X = X'$, this is **self attention**, otherwise it is **cross attention.**

Multi-head attention combines several such operations in parallel, and Y is the concatenation of the results along the feature dimension to which is applied one more linear transformation.

Left: self-attention, right: cross-attention.

Dimension of matrices in attention mechanism

$$X \in \mathbb{R}^{T \times D_{\text{in}}}, \qquad X' \in \mathbb{R}^{T' \times D_{\text{in}}}$$

$$W_Q \in \mathbb{R}^{D_{qk} \times D_{\text{in}}}, \qquad W_K \in \mathbb{R}^{D_{qk} \times D_{\text{in}}}$$

$$W_V \in \mathbb{R}^{D_v \times D_{\text{in}}}$$

$$Q = XW_Q^T \in \mathbb{R}^{T \times D_{qk}}$$

$$K = X'W_K^T \in \mathbb{R}^{T' \times D_{qk}}$$

$$V = X'W_V^T \in \mathbb{R}^{T' \times D_v}$$

$$A = \text{softmax}_{\text{row}}\left(QK^T/\sqrt{D_{qk}}\right) \in \mathbb{R}^{T \times T'}$$

$$Y = AV \in \mathbb{R}^{T \times D_v}$$

Some notes on dimension of query, key, value

- The query and key parameter matrices W_Q and W_K have the same dimension $(D \times D_{qk})$ because Q and K need to be compatible in scaled dot product.
- The value parameter matrix W_V has a different dimension from the key matrix $(D \times D_v)$.
- Where are the dimension comes from:
  - D_{qk}: determined by designer
  - D_v: determined by intermediate output (which is intermediate input of the downstream task).

Summary of attention layer in neural network

- Query (each t, each row) contain information of the current training examples, constructed by linear transformation of the input training example.
- Memory contain information of the references (other training examples) through keys and values, constructed by linear transformation of the input of reference.
- The attention weights in A is constructed from compatibility of query and keys through scaled dot product and softmax, which is non-linear function.
- The weight is used to to average the value matrix V in the memory for reference, from that the predicted values Y are constructed.

Varieties of attention mechanism

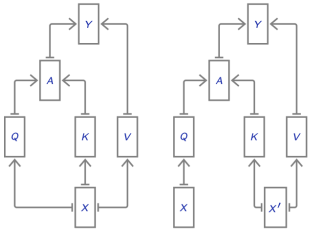
Attention mechanism can be categorized in different ways:

- By reference: self-attention vs cross-attention
- By masking: causal attention vs local attention vs full attention
- By number of heads: single-head attention vs multi-head attention
- By others?: the function for compatibility score, function to compute attention weight from compatibility score?

By reference

A standard attention layer takes as input two sequences X and X' and computes the tensors K, V, and Q as per-row linear functions.

$$Q = XW^{Q\top}$$
$$K = X'W^{K\top}$$
$$V = X'W^{V\top}$$
$$A = \text{softmax}_{row}\left(\frac{QK^\top}{\sqrt{D}}\right)$$
$$Y = AV$$



When $X = X'$, this is **self attention**, otherwise it is **cross attention.**

Multi-head attention combines several such operations in parallel, and Y is the concatenation of the results along the feature dimension to which is applied one more linear transformation.

By masking

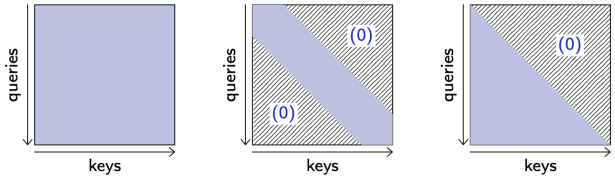
Varieties by masking (which to be NOT included in attention weight computation)

Masking is happen at the point of computing attention weights.

- Causal attention: masking future weights from computing attention in softmax
- Local attention: masking distant from computing attention
- Full attention: no masking in attention weight computation

By masking

It may be useful to mask the attention matrix, for instance in the case of self-attention, for computational reasons, or to make the model causal for auto-regression.



Full attention

Local attention

$|i-j| > \Delta \Rightarrow A_{i,j} = 0$

Causal attention

$j > i \Rightarrow A_{i,j} = 0$

Go to page 36

By number of heads

Varieties by number of attention heads

- Single-head attention
- In Multi-head attention:
  - Variation of head parameters comes from designers' choice of: hidden node size, weight initialization, learning rate, etc in each head.
  - Results of different heads are aggregated through concatenation

Multi-head attention

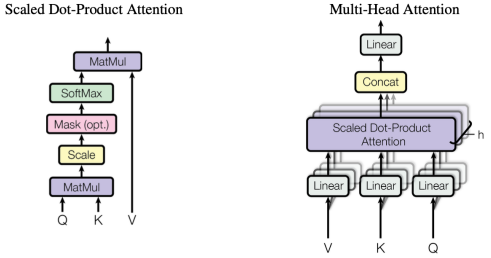


Figure 2: (left) Scaled Dot-Product Attention. (right) Multi-Head Attention consists of several attention layers running in parallel.

Source note: Source: Vaswani et al. (2017)

Permutation properties in attention mechanism

What is read in pure attention layer?

Given a permutation σ and a 2d tensor X, let us use the following notation for the permutation of the rows:

$$\sigma(X)_i = X_{\sigma(i)}.$$

The standard attention operation is invariant to a permutation of the keys and values:

$$Y(Q, \sigma(K), \sigma(V)) = Y(Q, K, V),$$

and equivariant (permute same way) to a permutation of the queries, that is the resulting tensor is permuted similarly:

$$Y(\sigma(Q), K, V) = \sigma(Y(Q, K, V)).$$

Consequently self attention and cross attention are equi-variant to permutations of X, and cross attention is invariant to permutations of X'.

Self-attention: why it is needed in text processing

- Self attention mechanism process a sequence with reference to that own sequence
- Example: when processing a sentence, each word in a sentence will be processed with reference to all other words in the sentence. After that, they are concatenated together to be a output sequence
- This allows flexible retrieval of information from a reference (memory) based on a query, which can capture long-range dependencies without loss of information.
- In self-attention, the training example itself serves as the reference, allowing each token to attend to all other tokens in the sequence, effectively capturing contextual relationships.

Self attention and context reading. Which part is read?

Checking your understanding of attention mechanism, what do you think the attention mechanism in pure attention layer is reading?

- The content of the word?
- The word order?
- The word within the context of other words?
- Back to the slide of properties of plain attention layer under permutation

Self attention and context reading. Which part is read?

- So far, plain attention layer applied to token levels does not account for the order of the word.
- How to account for word order: Positional encoding (see appendix)

Section 4: Self-Attention layer in processing sequence of tokens:

Self-attention vs recurrent and LSTM

Attention.

- Retrieving information with reference
- Information of the whole sequence in reference is designed to "equally" enter the explicit memory.[a]
- Long and complicated dependencies can be obtain as memory reference is available: exactly sequence $x_1, x_2, \ldots, x_T$.
- Overall information in V_j of reference input x_j for each query Q_i are retrieved with attention weight A_{ij}
- After the running through the query input $x_1, x_2, \ldots x_T$ with reference input $x_1, x_2, \ldots x'_T$, the attention mechanism can retrieve information of "well preserved" memory by directly and flexibly linking x_i with x_j through the non-linear constructed attention weight A_{ij}, and value information V_j.

Recurrent and LSTM.

- no reference/ explicit memory for retrieving.
- sequence dependencies (kind of implicit memory) can be partially preserve during the process that the recurrent input x_t and hidden state h_{t-1} constructs "memory-like" hidden state h_t through gating.
- This process with forget gate determined by $x\_$ leads to information loss as $h_t$only have the part of h_{t-2}that is not forgotten in h_{t-1} due to x_{t-1}.
- This forgetting is bring to the end of processing, state h_T
- Here is where the pattern of long-range and complex dependencies can be lost in RNN and LSTM.

[a] through just linear transformation to get K and V (learned matrix W_K and W_V is not equal weight. They was designed equally with same initialization)

Self-attention vs convolutional

Attention.

- Retrieving information with much more flexible to any location in the sequence, which is determined by the attention weight.

Convolutional.

- retrieve information of the sequence with a fixed window size, which is determined by the kernel size.

Self-attention in application: Transformer models

- Earlier strong sequence models used recurrent or convolutional networks, often with attention added.
- The Transformer removed recurrence and convolution from the main architecture and relied on attention mechanisms.
- Instead of single attention head, it used multi-head attention to capture different types of relationships.
- Attention moved from being an auxiliary alignment module to being the main representation-building operation.
- Transformer structure is provided in appendix

Section 5: Variational token length and fixed contextual synthesized vector:

5. Variational token length and fixed contextual synthesized vector

How to obtain a fixed contextual vector

- Variable sequence length- additional complexity in coding attention layer: Padding and masking in pooling.
- Output of plain self-attention layer is still a sequence of same length of the input sequence, which is variational across different examples.
- Pooling (mean, sum, etc.) can be used in this case of variational sequence length after pure attention layer to pre-attention input.
- Additional attention layer for pooling: structure attention pooling (see Lin et al., 2017)

Section 6: Toy codes and examples of application:

Toy code to use ready models in PyTorch

- What is needed to be written for a self-attention layer in PyTorch?
- The forward pass of the self-attention layer, which includes the conversion of input to query, key, value, and the computation of attention weights and output.
- No backward pass and explicit gradient computation is provided here because forward pass in enough when PyTorch has auto-grad.
- The code is adapted from Fleuret's handout with some modification of scale factor and masking. His original code in hand-out example is without scaled factor.
- These toy code is then run on simulated data set (see appendix)

Toy code of plain self attention layer

```
class SelfAttentionLayer(nn.Module):
    def __init__(self, D, D_v, D_qk):
        super().__init__()
        # x: N (batch size) x D (embedding dimension) x T (sequence length)
        # kernel_size=1: look at one position at a time as plain attention.
        # But attention process still use whole sequence.
        self.conv_Q = nn.Conv1d(D, D_qk, kernel_size=1, bias=False) # W_Q: D_qk x D,
        self.conv_K = nn.Conv1d(D, D_qk, kernel_size=1, bias=False) # W_K: D_qk x D,
        self.conv_V = nn.Conv1d(D, D_v,  kernel_size=1, bias=False) # W_V: D_v x D
        self.D_qk = D_qk
```

Source note: Code is adapted with help from Codex based on examples in Fleuret, Deep Learning Course, Handout 13.2, slide 18

Toy code of plain self attention layer: forward

```
def forward(self, x, mask=None):
    #mask: N x T, with 1/True for real tokens and 0/False for padding
    Q = self.conv_Q(x)  # Q: N x D_qk x T, not dimension in previous slide
                        # because in pytorch Conv1d for batching
                        # C_in = D, C_out = D_qk,

                        # L=T, L_out=T'=T as self attention.
                        # So the output of conv_Q will have shape (N, D_qk, T).
    K = self.conv_K(x)  # same reason, K: N x D_qk x T
    V = self.conv_V(x)  # same reason, V: N x D_v x T.
    # Due to switching D and T position in tensor, transpose is needed in computation:
    # Mathematically this is still QK^T
    scores = Q.transpose(1, 2).matmul(K)   # N x T x T
    scores = scores / math.sqrt(self.D_qk)
    # scale the scores to prevent them from being too noisy in softmax

    # Extra padding mask might be needed for variational sequence length
    # Each row/query softmax should ignore padded ones.
    # Setting their score to -inf makes softmax assign weight 0 to padding.
    if mask is not None:
        scores = scores.masked_fill(
            mask[:, None, :] == 0,
            float("-inf")
        )
    A = F.softmax(scores, dim=2)  # row softmax, dimension N x T x T
    y = A.matmul(V.transpose(1, 2)).transpose(1, 2)  # N x D_v x T
    return y, A
```

Hugging Face pre-trained transformer

- Use a Hugging Face pre-trained transformer to obtain contextualized token vectors
- The model to be used is a light one: all-MiniLM-L6-v2
- detail of the model can be found at `https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2`
- How to run:

```
texts = ["This is a sentence to test the Hugging Face pre-trained transformer.", "Here is a text chunk. It is
↪  to examine how good the transformer in the Hugging Face library is!"]
pip install sentence-transformers
from sentence_transformers import SentenceTransformer
model = SentenceTransformer("sentence-transformers/all-MiniLM-L6-v2")
embeddings = model.encode(texts) # out put is of shape (2, 384) in this case.
```

  Correlation of embedding representation for the two text: 0.7415735721588135

- Next: example of using this pre-trained transformer for some posts of Presidents Trump on Truth. The similarity is proxied by correlation coefficient. Data is from https://trumpstruth.org. The data is accessed on June 11th, 2026 at 06:47 pm. Code for downloading the data is generated with my prompt in Codex.

A pair of Truth posts from President Trump

Text 1
Last month, I directed our Great U.S. Military to execute a secret mission to support Oil Tankers and other Commercial Ships through the Straight of Hormuz. Today, I am pleased to announce that this effort has resulted in more than 100 MILLION Barrels of Oil making its way through the Straight, and into the Open Market. More than 200 Commercial Ships have safely traveled through the Strait. This wildly successful effort is because the UNITED STATES of AMERICA CONTROLS the Strait of Hormuz – NOT Iran. Their military is defeated, and their economy is lost. It is over for Iran! Thank you for your attention to this matter. President DONALD J. TRUMP
Correlation coefficient: 0.627

Text 2
The Fake News Media refuses to report how EFFECTIVE the U.S. Naval BLOCKADE is, the most successful Blockade in the history of Naval Warfare. NOTHING GETS THROUGH unless we want it to. IT IS A STEEL WALL! Iran is doing ZERO business, not paying their military, or any of their bills, and quickly becoming a FAILED NATION! Lots of oil is getting out. Praise be to Allah! President DONALD J. TRUMP

A pair of Truth posts from President Trump

Text 1

In a reversal, Mar-a-Lago helipad to remain past end of Trump's term: https://www.palmbeachpost.com/story/news/trump/2026/06/09/in-a-reversal-mar-a-lago-helipad-to-remain-past-end-of-trumps-term-palm-beach-florida/90455786007/

Correlation coefficient: 0.069

Text 2

Wow! CITI was ranked Number 1 in topping M&A Advisory Market by Value in Q1. Congratulations to Jane F and ALL of her great people. They have worked really hard! BIG comeback for CITI!!! President DONALD J. TRUMP

Section 7: Summary:

Main takeaways

- Tokenization and embedding are necessary to make text data numerical.
- Attention mechanism is a soft search process with query, key, value, and reference/ memory. It can be implemented as a layer in neural network with learnable parameters for query, key, value construction.
- Self-attention mechanism is when input for query is same as input for key and value
- Self-attention help to computes context-dependent weighted averages. Context can be gathered from any positions in a more flexible and complex dependencies.
- Self attention does not account for word order, but this can be solved by positional encoding.
- Variational sequence length can be handled with pooling, including attention pooling to obtain a single fixed-length vector.

Thank you

Questions?

Text Processing with Attention Mechanism

References I


Fleuret, F. *Deep Learning: 12.3 Word Embeddings and Translation*. University of Geneva deep learning course handout.


Fleuret, F. *Deep Learning: 13.1 Attention for Memory and Sequence Translation*. University of Geneva deep learning course handout.


Fleuret, F. *Deep Learning: 13.2 Attention Mechanisms*. University of Geneva deep learning course handout.


Fleuret, F. *Deep Learning: 13.3 Transformer Networks*. University of Geneva deep learning course handout.


Lin, Z., Feng, M., dos Santos, C. N., Yu, M., Xiang, B., Zhou, B., and Bengio, Y. (2017). A Structured Self-attentive Sentence Embedding. *arXiv preprint arXiv:1703.03130*. `https://arxiv.org/abs/1703.03130`


Vaswani, A., Shazeer, N., Parmar, N., Uszkoreit, J., Jones, L., Gomez, A. N., Kaiser, L., and Polosukhin, I. (2017). Attention is all you need. *NeurIPS*. `https://proceedings.neurips.cc/paper_files/paper/2017/file/3f5ee243547dee91fbd053c1c4a845aa-Paper.pdf`

References II


Mitchell, Melanie. Artificial Intelligence: A Guide for Thinking Humans. New York: Farrar, Straus and Giroux, (2019)

Acknowledgment and AI Assistance Disclosure

I deeply appreciate the researchers, instructors, and authors whose work on attention mechanisms and deep learning made this learning project possible.

Code examples are adapted from François Fleuret's course materials.

OpenAI Codex was used to assist with code adaptation and slide formatting. All explanations, verification, educational interpretation, and final presentation framing are my own.

Some concepts linked with previous presentations

This will be helpful for formulation and attention mechanism.

- Autoencoder: token, encoder, decoder, embedding.
- Recurrent neural network and LSTM: how to formulate and process sequences, recurrent input and memory,
- Autoregression: masking, later used for different varieties of attention mechanisms.
- Others links? Your recommendation along this presentation.

Neural Turing Machines: explicit memory with attention in reading and writing

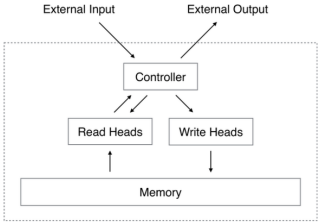
Skipped now, will be back to it if questioned or if I have time.

- Graves et al. (2014): equip a deep model with an explicit memory M to allow for long-term storage and retrieval rather than just c_t after gating in LSTM.
- Contains two basic components: a neural network controller and a memory bank.
- The controller interacts externally via intermediate input and output vectors.
- Unlike a standard network, it also interacts with a memory matrix using selective read and write operations (attention weights come here).
- The network outputs that parametrize these operations are called "heads."
- Their main use of this mechanism nowadays is to provide long-term dependency.

Source note: Source: François Fleuret, Deep Learning Course, Handout 13.1, slide 5.

Appendix: memory attention in Neural Turing Machines

Graves et al. (2014) proposed to equip a deep model with an explicit memory to allow for long-term storage and retrieval.



(Graves et al., 2014)

Neural Turing Machines: additional memory state

For memory: Hidden internal state

$$M_t \in \mathbb{R}^{N \times M}$$

where t is the time step, N is the number of entries in the memory and M is their dimension.

A "controller" is implemented as a standard feed-forward or recurrent model and at every iteration t it computes activations that modulate the reading / writing operations.

Neural Turing Machines: reading and writing

- Reading, where given attention weights $w_t \in \mathbb{R}_+^N$, $\sum_n w_t(n) = 1$, it gets reading

$$r_t = \sum_{n=1}^{N} w_t(n) M_t(n).$$

- Writing, where given attention weights w_t, an erase vector $e_t \in [0,1]^M$ and an add vector $a_t \in \mathbb{R}^M$, the memory is updated with

$$\forall n, \quad M_t(n) = M_{t-1}(n)(1 - w_t(n)e_t) + w_t(n)a_t.$$

The controller has multiple "heads", and computes at each t, for each writing head w_t, e_t, a_t, and for each reading head w_t, and gets back a read value r_t.
This early use of attention treated weights as a differentiable way to read from and write to memory.

The single state bottleneck in seq2seq

Not a problem directly in my project, seq2seq is like translation. But it is main motivation for the nowadays popular use of attention mechanism for dependencies.

Encoder recurrence

$$h_t = f(x_t, h_{t-1}), \qquad v = h_T.$$

x_t is the input at time t, h_t is the hidden state at time t, and v is the final state that summarizes the whole input sequence.

Decoder

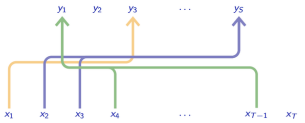
$\hat{y}_t$ is generated from $p(\hat{y}_1, \ldots, \hat{y}_{t-1}, v)$.

The single state bottleneck in seq2seq

- The final state v must carry all relevant information from the input sequence (last state h_T).
- This creates a capacity bottleneck when sentence structure are long and complex.
- Because the decoder looks only through v instead of $h_1 \dots h_T$, it is hard to learn long-term dependencies.
- In attention-based seq2seq, similar to memory M in Turing machine, a fixed memory made from encoder states $h_1, \dots, h_T$, and the decoder reads from it using attention weights.

Attention vs recurrent in seq2seq 1

Attention mechanisms (Bahdanau et al., 2014) can transport information from parts of the signal to other parts **specified dynamically**.

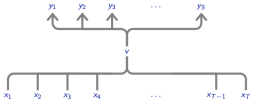


Notes

This diagram illustrates the core idea of the attention mechanism: there are direct flows of information between parts of the sequence which are dynamically identified from the activations.

Attention vs recurrent in seq2seq 2

The main weakness of such an approach is that all the information has to flow through a single state v, whose capacity has to accommodate any situation.



There are no direct "channels" to transport local information from the input sequence to the place where it is useful in the resulting sequence.

Notes

On the diagram, $x_1, \ldots, x_T$ is the input sequence, for instance a sentence in German. $y_1, \ldots, y_S$ is the output sequence, for instance a sentence in English.
With a recurrent model, all the information flows through a single state v, which is obtained after visiting the full input sequence. Therefore, v has to contain all the information required to generate the $y_1, \ldots, y_S$.

Why position must be added

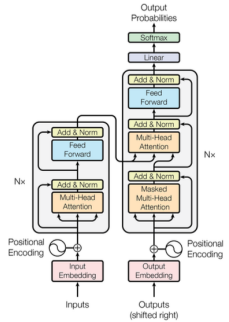
- Attention is invariant to permutation of keys and values.
- This is useful when content matters, but it means that absolute order is not automatically encoded.
- For text, word order can change meaning; therefore the model needs positional information.

Transformer positional encoding

$$PE_{(pos,2i)} = \sin\left(pos/10000^{2i/d_{model}}\right),$$

$$PE_{(pos,2i+1)} = \cos\left(pos/10000^{2i/d_{model}}\right).$$

How Transformer is structured



"Original" Transformer (Vaswani et al., 2017).

Potential use of transformer for my practical application in reading context from text chunks

- The original Transformer was designed for sequence-to-sequence translation with both encoder and decoder.
- For representation learning, an encoder-only structure is enough: it maps an input token sequence to contextual token vectors.

Structure attention pooling

Published as a conference paper at ICLR 2017

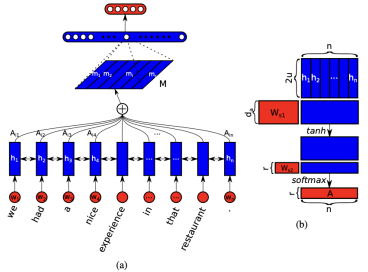


Figure 1: A sample model structure showing the sentence embedding model combined with a fully connected and softmax layer for sentiment analysis (a). The sentence embedding M is computed as multiple weighted sums of hidden states from a bidirectional LSTM $(\mathbf{h_1}, ..., \mathbf{h_n})$, where the summation weights $(A_{i1}, ..., A_{in})$ are computed in a way illustrated in (b). Blue colored shapes stand for hidden representations, and red colored shapes stand for weights, annotations, or input/output.

Source note: Source: Lin et al. (2017)